



Egyptian Clinic AI Voice Agent

End-to-End System Design Document

Prepared by: **Nour Eldin Ayman Galal**

Ain Shams University — Faculty of Computer & Information Sciences

Version 1.0 | 2025 | Portfolio Project

Component	Technology
Voice STT	Whisper large-v3 (Arabic)
Agent Framework	LangGraph (Multi-Agent)
LLM Brain	GPT-4o mini + GPT-4o (Two-Layer)
Custom TTS	SparkTTS Fine-tuned (Egyptian Arabic)
Database	PostgreSQL + Supabase
Notifications	WhatsApp Business API + SMS
Backend	FastAPI (Python)
Scheduler	Celery + Redis

1

Project Overview

1.1 Problem Statement

Egyptian clinics rely heavily on manual phone calls for appointment management. Receptionists spend the majority of their working day answering repetitive calls, leading to human errors, missed appointments, and frustrated patients.

- High workload on clinic receptionists — repetitive calls all day
- Long patient wait times on the phone
- Human errors in scheduling and patient data entry
- No automated reminders — high no-show rates
- No 24/7 availability for patient inquiries

1.2 Solution

A three-agent AI platform that automates the full patient lifecycle — from the initial phone call to post-visit analytics — entirely in Egyptian Arabic dialect.

Agent	Role	Triggered By	Output
Agent 1 — Voice Receptionist	Handles all patient calls	Incoming phone call	Booked/changed/cancelled appointment
Agent 2 — Doctor Recording	Records procedures after visit	Doctor voice command	Structured procedure + price in DB
Agent 3 — Analytics Engine	Monthly business intelligence	Celery scheduler (1st of month)	Dashboard + PDF report for doctor

1.3 Project Rating

Dimension	Score	Justification
Originality	8 / 10	Egyptian clinic context — unique and locally relevant
Technical Depth	9.5 / 10	Three agents + custom TTS + voice pipeline + analytics
Real World Value	9 / 10	Solves a genuine pain point in the Egyptian market
Portfolio Impact	9.5 / 10	Multi-agent system — rare at junior level
Overall	9.2 / 10	Strong junior AI engineer portfolio project

2 System Architecture

2.1 Full Three-Agent Pipeline



[illegible]

2.2 Two-Layer Brain (Cost Efficiency)

Layer	Model	Handles	Cost/Call
Layer 1 — Router	GPT-4o mini	90% of calls — routine bookings, cancellations, lookups	~\$0.001
Layer 2 — Resolver	GPT-4o	10% of calls — ambiguous, angry patients, edge cases	~\$0.010

Cost Efficiency
100 calls/day — GPT-4o only: \$1.00/day = \$30.00/month
100 calls/day — Two-Layer: \$0.19/day = \$5.70/month
Savings: ~81% cost reduction with identical quality output

3 Technology Stack

Layer	Technology	Purpose	Why This Choice
Telephony	Twilio Voice	Inbound call handling + webhooks	Best docs, Arabic number support
STT	Whisper large-v3	Arabic + Egyptian dialect transcription	Best open-source Arabic STT available
Agent Framework	LangGraph	Stateful multi-step agent orchestration	Best for complex conversation state
LLM Layer 1	GPT-4o mini	Fast routing + simple intent handling	Fast, cheap, strong Arabic understanding
LLM Layer 2	GPT-4o	Complex reasoning + edge cases	Best Arabic tool-calling performance
TTS	SparkTTS (fine-tuned)	Egyptian Arabic voice output	LLM-based — language-agnostic, fine-tunable
Database	PostgreSQL + Supabase	All clinic data storage	Free tier, real-time, built-in REST API
Scheduler	Celery + Redis	Reminders + monthly analytics trigger	Reliable async task queue
Notifications	WhatsApp Business API	Appointment reminders + location	Primary messaging platform in Egypt
SMS Fallback	Twilio SMS	Patients without WhatsApp	Covers elderly patients
Backend	FastAPI	API layer connecting all components	Async, fast, already in your stack
Frontend	React + Recharts	Doctor dashboard + analytics charts	Standard, fast to build
Hosting	Railway / Render	Backend + scheduler deployment	Easy deploy, free tier available

4

Database Schema

4.1 Core Tables

```
-- PATIENTS
CREATE TABLE patients (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  phone VARCHAR(20) UNIQUE NOT NULL,
  age INTEGER,
  gender VARCHAR(10),
  medical_notes TEXT,
  language_pref VARCHAR(10) DEFAULT 'arabic',
  whatsapp_opt_in BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
-- DOCTORS
CREATE TABLE doctors (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  specialty VARCHAR(100),
  working_hours JSONB, -- { "Sun": ["09:00","17:00"], ... }
  is_active BOOLEAN DEFAULT TRUE
);
```

```
-- APPOINTMENTS
CREATE TABLE appointments (
  id SERIAL PRIMARY KEY,
  patient_id INTEGER REFERENCES patients(id),
  doctor_id INTEGER REFERENCES doctors(id),
  scheduled_at TIMESTAMP NOT NULL,
  status VARCHAR(20) DEFAULT 'pending',
  -- pending | confirmed | done | cancelled | no_show
  reminder_sent BOOLEAN DEFAULT FALSE,
  call_transcript TEXT,
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

4.2 Procedures & Analytics Tables (NEW)

```
-- PROCEDURES CATALOGUE (clinic-specific)
CREATE TABLE procedures (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100), -- "Root Canal", "Crown", "Dental X-ray"
  category VARCHAR(50), -- restorative | cosmetic | preventive
  base_price DECIMAL(10,2),
  duration_mins INTEGER
);

-- WHAT EACH PATIENT HAD DONE
CREATE TABLE patient_procedures (
  id SERIAL PRIMARY KEY,
  appointment_id INTEGER REFERENCES appointments(id),
  patient_id INTEGER REFERENCES patients(id),
  procedure_id INTEGER REFERENCES procedures(id),
  quantity INTEGER DEFAULT 1,
  price_charged DECIMAL(10,2),
  tooth_area VARCHAR(50), -- "Upper Front", "Lower Back"
  notes TEXT,
  performed_at TIMESTAMP
);

-- PAYMENTS
CREATE TABLE payments (
  id SERIAL PRIMARY KEY,
  patient_id INTEGER REFERENCES patients(id),
  appointment_id INTEGER REFERENCES appointments(id),
  amount DECIMAL(10,2),
  method VARCHAR(20), -- cash | card | installment
  status VARCHAR(20), -- paid | partial | pending
  paid_at TIMESTAMP
);

-- CALL LOGS
CREATE TABLE call_logs (
  id SERIAL PRIMARY KEY,
  patient_id INTEGER REFERENCES patients(id),
  call_duration INTEGER,
  transcript TEXT,
  intent_detected VARCHAR(50),
  outcome VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);
```

4.3 Race Condition Prevention

```
-- Atomic slot booking – prevents double-booking
BEGIN;
SELECT id FROM appointments
WHERE doctor_id = $1
AND scheduled_at = $2
AND status != 'cancelled'
FOR UPDATE; -- Row-level lock

INSERT INTO appointments (patient_id, doctor_id, scheduled_at, status)
VALUES ($3, $1, $2, 'confirmed');
COMMIT;

-- On conflict: return alternative available slots to agent
```


1 Agent 1 — Voice Receptionist

5.1 LangGraph Agent State

```
class ClinicAgentState:
    patient_phone: str # caller ID from Twilio
    patient_data: dict # retrieved or newly created
    is_new_patient: bool # registration needed?
    conversation_history: list # full call transcript
    current_intent: str # booking|cancel|reschedule|query
    collected_slots: dict # date, time, doctor preference
    confirmation_pending: bool # awaiting patient confirmation
    language_preference: str # arabic | english
    escalated: bool # sent to Layer 2?
    call_summary: str # auto-saved after call ends
```

5.2 Available Tools

Tool	Description	DB Operation
lookup_patient(phone)	Find patient by phone number	SELECT from patients
register_patient(name, age, phone)	Create new patient record	INSERT into patients
get_available_slots(doctor_id, date)	Return open appointment slots	SELECT from appointments
book_appointment(patient_id, doctor_id, datetime)	Book a new appointment	INSERT with row lock
reschedule_appointment(appointm ent_id, new_datetime)	Change appointment time	UPDATE appointments
cancel_appointment(appointment_id)	Cancel an appointment	UPDATE status=cancelled
send_whatsapp_confirmation(pati ent_id, appointment_id)	Send booking confirmation	External API call

5.3 Conversation Flows

Flow 1 — New Patient Books Appointment

```
TRIGGER: Incoming call from unknown number

1. Agent greets in Egyptian Arabic
2. lookup_patient(phone) → NOT FOUND
```

3. Agent: "██████ █ ██████ █████ █████ █████"
4. Collect: name, age, gender
5. register_patient() → new record in DB
6. Agent: "████ █████ █████ █████ ██ █████"
7. Collect: doctor preference, date, time
8. get_available_slots(doctor_id, date)
9. Agent offers available options
10. Patient confirms choice
11. book_appointment() → DB write with row lock
12. WhatsApp confirmation sent
13. call_log saved with full transcript

Flow 2 — Existing Patient Reschedules

1. lookup_patient(phone) → FOUND
2. Agent: "■■■■■ ■■ [name]■ ■■■■ ■■■■"
3. Patient: "■■■■ ■■■■ ■■■■■■ ■■■■"
4. get_patient_appointments(patient_id)
5. Agent confirms which appointment to reschedule
6. Collect new preferred date/time
7. get_available_slots() → offer options
8. Patient confirms new slot
9. reschedule_appointment() → DB update
10. WhatsApp confirmation sent

Flow 3 — Day-Before Reminder (Automated)

```

TRIGGER: Cerebrum Beat – runs daily at 9:00 AM

SELECT * FROM appointments

WHERE DATE(scheduled_at) = CURRENT_DATE + 1

AND status = 'confirmed'

AND reminder_sent = FALSE

For each appointment:

IF whatsapp_opt_in = TRUE:

→ Send WhatsApp with: doctor name, time,
Google Maps clinic link

ELSE:

→ Send SMS fallback (Twilio)

UPDATE appointments SET reminder_sent = TRUE

```

2

Agent 2 — Doctor Recording Agent

After each patient appointment the doctor speaks naturally about what was done. Agent 2 transcribes, extracts structured data, calculates the price, confirms with the doctor, and writes to the database — all within seconds.

6.1 Full Processing Pipeline

Doctor Recording Flow
STEP 1 — Doctor speaks after appointment finishes:
"■■■■■ ■■■■ ■■■■ ■■■■■■ ■■■■■■ ■■■■"
STEP 2 — Whisper large-v3 transcribes the voice input
STEP 3 — NLP Extraction Agent (GPT-4o) parses the text:
{ patient: "■■■■■", procedures: [■■■■■ x2, ■■■■■■ ■■■■ x1] }
STEP 4 — Pricing Engine calculates total:
■■■■■ x2 = 400 EGP
■■■■■■■ ■■■■ x1 = 150 EGP
Total = 550 EGP
STEP 5 — Agent reads back to doctor by voice:
"■■■■■■■ ■■■■ ■■■■ ■■■■ ■■■■■■ ■■■■ 400 ■■■■■■ ■■■■ ■■■■ 150
■■■■■■■■■ 550 ■■■■ — ■■■■■■"
STEP 6 — Doctor says "■■■ ■■■■■" → DB write ■
patient_procedures + payments tables updated
Analytics agent now has clean structured data

6.2 NLP Extraction Prompt Design

SYSTEM PROMPT (Agent 2):

■■■■

```

You are a dental procedure extractor for an Egyptian clinic.
The doctor will describe procedures in Egyptian Arabic dialect.
Extract ONLY valid JSON – no explanation, no markdown.

Input: "████ ███ ███ █████████ █████████ ███"
Output: {
  "patient_name": "████",
  "procedures": [
    { "name": "████", "quantity": 2, "tooth_area": "██████" },
    { "name": "██████ ███", "quantity": 1, "tooth_area": null }
  ],
  "ambiguous": false,
  "clarification_needed": null
}

If patient name is missing → set ambiguous: true
If procedure is unclear → set clarification_needed: "question"
"""

```

6.3 Pricing Engine

```

# Dental procedures price catalogue
PROCEDURES = {
  "███": { "base_price": 200, "unit": "per_tooth" },
  "██████ ███": { "base_price": 150, "unit": "per_session" },
  "██████": { "base_price": 800, "unit": "per_tooth" },
  "████": { "base_price": 150, "unit": "per_tooth" },
  "████ ███ ███": { "base_price": 400, "unit": "per_tooth" },
  "████ ████████████": { "base_price": 200, "unit": "per_session" },
  "██████ ███": { "base_price": 600, "unit": "per_tooth" },
  "██████": { "base_price": 300, "unit": "per_month" },
  "██████": { "base_price": 1200, "unit": "per_tooth" },
}

def calculate_total(procedures: list) -> tuple[int, list]:
  total, breakdown = 0, []
  for proc in procedures:
    unit_price = PROCEDURES[proc["name"]]["base_price"]
    line_total = unit_price * proc["quantity"]
    total += line_total
    breakdown.append({
      "procedure": proc["name"],
      "quantity": proc["quantity"],

```

```
"unit_price": unit_price,
"total": line_total
})
return total, breakdown
```

6.4 Ambiguity Handling

Doctor Says	Agent Response	Action
"[redacted] [redacted] [redacted] [redacted]"	Clear — extract immediately	Calculate + confirm
"[redacted] [redacted]" (no patient name)	"[redacted] [redacted] [redacted] [redacted] [redacted]"	Ask for clarification
"[redacted] [redacted] [redacted] [redacted]"	"[redacted] [redacted] [redacted] [redacted] [redacted]"	Ask for procedure name
"[redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted]"	"[redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted]"	Ask for quantity

3 Agent 3 — Monthly Analytics Engine

On the 1st of every month, Celery triggers the analytics agent. It pulls all procedures, payments, and appointment data, reasons over it with GPT-4o, generates Arabic-language insights, updates the React dashboard, and emails a PDF report to the doctor.

7.1 Analytics Agent Workflow

```

TRIGGER: Celery Beat – 1st of every month at 08:00 AM

Step 1 → Pull last month's data from DB
appointments, patient_procedures, payments, patients

Step 2 → Run analytics tools
generate_revenue_summary(month, year)
identify_top_procedures(month, year)
flag_inactive_patients(months_threshold=6)
calculate_no_show_impact(month, year)
compare_to_previous_month()
get_new_vs_returning_ratio(month, year)

Step 3 → GPT-4o reasons over aggregated data
Generates 3-5 Arabic-language business insights

Step 4 → Update React dashboard (charts auto-refresh)

Step 5 → Generate PDF monthly report → email to doctor

Step 6 → Store analytics snapshot for trend comparison

```

7.2 Dashboard Metrics

Section	Metrics Shown
Revenue Overview	This month total vs last month growth % daily trend chart
Patient Insights	Total active new this month returning inactive 6+ months
Top Procedures	Most performed procedures ranked by count and revenue generated
Operational Metrics	Busiest day/hour no-show rate no-show revenue lost avg per visit
AI Insight (Agentic)	GPT-4o generated Arabic recommendation for the doctor
Per-Patient History	Full procedure history, total spent, last visit, payment status

7.3 AI-Generated Insight Example

Sample Monthly AI Insight (Arabic)

"[...] 28% [...]".

■■ ■■■■■ ■■■■■■■ ■■■■■■■■ 3,600 ■■■■ ■■ ■■■■■.

[illegible]

"█████ 3 ██████████ ███████ ███████ ███: ████████ (14,000 EGP) | █████ (9,000 EGP) | ████████ (6,200 EGP)"

[illegible]

5 Custom Egyptian Arabic TTS (SparkTTS)

8.1 Why SparkTTS for Arabic

Aspect	Traditional TTS	SparkTTS (LLM-based)
Architecture	Phoneme rules + acoustic model	LLM token prediction backbone
New Language	Requires phoneme table rebuild	Fine-tune on new language data
Egyptian Dialect	Cannot adapt — hard-coded	Learns dialect prosody from examples
Filler Words	Fails on █████ / ███ / █████	Learns natural Egyptian rhythm
Fine-tuning Cost	Requires phoneme layer rebuild	Standard LLM fine-tuning on Colab

8.2 Dataset Creation Strategy

[illegible]

8.3 Fine-tuning Pipeline

Step 1 – Data Collection (Hybrid approach)

Option B: Scrape Egyptian Arabic YouTube + Whisper transcription

Recommended: Both combined

Step 2 – Audio Preprocessing

```
ffmpeg: resample to 22050Hz + normalize loudness
```

```
webrtcvad: remove silence segments
```

Montreal Forced Aligner: align audio to text

Step 3 – Tokenizer Extension

Extend SparkTTS tokenizer with Arabic Unicode block

Verify: Arabic chars tokenize correctly – not as [UNK]

Step 4 – Fine-tuning (Unsloth on Google Colab A100 – FREE)

Base: SparkAudio/Spark-TTS-0.5B checkpoint

Train on Egyptian clinic dataset

Estimated: 10-20 GPU hours

Step 5 – Evaluation

MOS score (human rating 1-5)

Compare against Azure ar-EG baseline

Publish model + dataset on HuggingFace

4 Agent 4 — WhatsApp Booking Agent

Why WhatsApp Booking?

A significant portion of Egyptian patients — especially women, introverts, elderly, and younger patients — feel shy or anxious about making phone calls to clinics. WhatsApp text booking removes this barrier entirely, meeting patients where they already spend their time. It also enables booking outside clinic hours with zero additional staff cost.

This is not just a convenience feature — it is a patient acquisition channel.

9.1 How It Fits Into the System

Agent 4 runs as a parallel channel alongside Agent 1. Both share the same tools, the same database, and the same business logic — the only difference is the interface: text instead of voice.

Aspect	Agent 1 (Voice)	Agent 4 (WhatsApp)
Interface	Phone call via Twilio	WhatsApp messages via WhatsApp Business API
STT / TTS	Whisper + SparkTTS	Not needed — pure text
Patient type	Comfortable with calls	Shy / prefer typing / outside hours
Response speed	Real-time (seconds)	Async (seconds to minutes)
Shared tools	lookup_patient, book, reschedule, cancel	Identical — same FastAPI tool layer
Shared DB	PostgreSQL (Supabase)	Identical — same tables
LLM Brain	LangGraph + GPT-4o mini	LangGraph + GPT-4o mini (same)

9.2 Technical Implementation

```

TRIGGER: Patient sends WhatsApp message to clinic number
"████ ██████████"

Step 1 — WhatsApp Business API webhook fires
POST /api/whatsapp/incoming
Payload: { from: "201XXXXXXXX", body: "████ ██████████" }

Step 2 — Extract phone number from WhatsApp ID
```

```
lookup_patient(phone) → FOUND or NEW

Step 3 – LangGraph agent processes message
Same agent state as Agent 1:
current_intent = "booking"
channel = "whatsapp" ← only difference

Step 4 – Agent replies via WhatsApp text
"■■■■■ ■■ [name]! ■■■■ ■■■■ ■■ ■■■■ ■■■■■■"

Step 5 – Multi-turn text conversation
Patient replies → agent reads → collects slots
date, time, doctor preference

Step 6 – Show available slots as WhatsApp List Message
(interactive buttons – see 9.3)

Step 7 – Patient taps their preferred slot

Step 8 – book_appointment() → DB write with row lock

Step 9 – Confirmation message sent:
"■■ ■■■■■ ■■ [name]! ■■■■ ■■■■ ■■■■■■ 15/8 ■■■■■■ 3",
+ Google Maps location link
+ "■■ ■■■■ ■■■■ ■■ ■■■■ ■■■■■■ ■■■■"
```

9.3 WhatsApp Interactive Messages

WhatsApp Business API supports interactive UI elements — use these to make booking feel like an app, not a chat:

Message Type	Used For	Example
List Message	Show available time slots	"■■■■■ ■■■■■■■■ ■■■■■■■■:" + list of slots as tappable rows
Quick Reply Buttons	Yes/No confirmation	"■■■■■ ■■■■■■■■" + [■■■■■ ■] [■■■■■ ■]
Quick Reply Buttons	Reschedule or cancel	[■■■ ■■■■■■] [■■■■ ■■■■■■] [■■■■■]
Template Message	Reminder (T-1 day)	Pre-approved template with name, time, Maps link
Plain Text	General conversation	All other agent responses

9.4 Full WhatsApp Conversation Example

```
Patient: "■■■■ ■■■■ ■■■■"

Agent: "■■■■■! ■■■■ ■■■■. ■■■■ ■■■■ ■■ ■■■■ ■■■■■■
[ ■■■■ ■■■■ - ■■■■ ] [ ■■■■ ■■■■ - ■■■■ ]"
```

```
Patient: [taps "0000 0000 - 0000"]

Agent: "0000! 0000 0000 00000000:"
[ 00000000 10 000000 ]
[ 00000000 2 00000 ]
[ 000000 4 00000 ]

Patient: [taps "000000 4 00000"]

Agent: "000 0000:"
000000 14/8 000000 4 00000
00 00000 0000 - 00000
[ 00000 0 ] [ 00000 0 ]"

Patient: [taps "00000 0"]

Agent: "00 00000 00 0000! 0
0 000000 14 00000 - 000000 4 00000
0000 00000 0000
0 [Google Maps link]
000000 000 000000 0000. 00 00000 0000 00000 000."

```

9.5 New Patient via WhatsApp

```
Patient sends first message — phone NOT in DB

Agent: "0000! 0000 000 0000000 00000 00000.
0000 0000 0000000"

Patient: "0000 000"

Agent: "00000 0000! 0000 000 000 00 00000"

Patient: "28"

Agent: "0000! 000000 00000.
000000 0000 0000 00000 [ 00 ] [ 00 ]"

→ register_patient(name="0000 000", age=28, phone=from_number)
→ Continue to booking flow

```

9.6 WhatsApp Channel — Key Advantages

Advantage	Details
24/7 availability	Patients book at midnight — agent replies instantly, no staff needed
No call anxiety	Shy patients, women calling about sensitive specialties, introverts
Message history	Patient can scroll back and see their appointment details anytime

Rich UI	Buttons and lists feel like a native app — no typing required for choices
Re-engagement	Inactive patient messages → agent can proactively start conversation
Proof of booking	Patient has written confirmation in their chat — reduces disputes
Shared infrastructure	Reuses 100% of existing tools and DB — minimal extra dev work

9.7 WhatsApp vs Voice — When Each Is Used

Channel Selection Logic
Voice (Agent 1):
→ Patient calls the clinic phone number
→ Elderly patients more comfortable with calls
→ Urgent same-day appointment needed
→ Patient needs to explain symptoms verbally
WhatsApp (Agent 4):
→ Patient messages the clinic WhatsApp number
→ Patient is shy / in a meeting / cannot speak
→ Outside clinic working hours
→ Patient wants written confirmation
→ Younger patients (prefer texting over calling)
Both channels → Same DB → Same appointment queue → Same dashboard

9.8 New API Endpoints for WhatsApp

Method	Endpoint	Description
POST	/api/whatsapp/incoming	WhatsApp webhook — receives patient messages
POST	/api/whatsapp/send	Send WhatsApp reply (text / list / buttons)
POST	/api/whatsapp/template	Send approved template (reminders, confirmations)
GET	/api/whatsapp/sessions/{phone}	Get active conversation session for a patient

9.9 Updated Agent Count Summary

Agent	Channel	Triggered By	Key Output
Agent 1 — Voice Receptionist	Phone call	Incoming Twilio call	Appointment booked via voice
Agent 2 — Doctor Recording	Doctor mic	Post-appointment voice command	Procedures + prices in DB
Agent 3 — Analytics Engine	Internal	Celery (1st of month)	Dashboard + PDF report
Agent 4 — WhatsApp Booking	WhatsApp	Patient message	Appointment booked via text

6

API Design (FastAPI)

Method	Endpoint	Description
POST	/api/voice/incoming	Twilio webhook — handle incoming patient call
POST	/api/voice/response	Return TTS audio response to caller
POST	/api/doctor/recording	Doctor voice recording → procedure extraction
POST	/api/whatsapp/incoming	WhatsApp webhook — receive patient messages
POST	/api/whatsapp/send	Send WhatsApp reply (text / list / buttons)
POST	/api/whatsapp/template	Send approved template message
GET	/api/patients/{phone}	Lookup patient by phone number
POST	/api/patients	Register new patient
GET	/api/appointments/today	All appointments for today (dashboard)
GET	/api/appointments/slots	Available slots for doctor/date
POST	/api/appointments	Book new appointment
PATCH	/api/appointments/{id}	Reschedule appointment
DELETE	/api/appointments/{id}	Cancel appointment
PATCH	/api/appointments/{id}/status	Update status: done / no_show
GET	/api/analytics/monthly	Monthly analytics data for dashboard
GET	/api/analytics/patient/{id}	Full procedure history for one patient
POST	/api/procedures	Add new procedure record manually
GET	/api/procedures/catalogue	Get all procedures and base prices

7 Key Challenges & Solutions

Challenge	Severity	Solution
Egyptian Arabic Dialect STT	9/10	Whisper large-v3 — best open-source Arabic dialect support
Voice Call Latency	8/10	Stream TTS + GPT-4o mini for fast Layer 1 routing
SparkTTS Tokenizer (Arabic)	7/10	Extend tokenizer with Arabic Unicode block before fine-tuning
Doctor Recording Ambiguity	7/10	Agent asks clarification before any DB write; always confirm
Concurrent Slot Booking	5/10	PostgreSQL SELECT FOR UPDATE row-level locking
Procedures Data Quality	6/10	Voice recording flow + confirmation step ensures clean data
Multi-turn Conversation State	7/10	LangGraph state machine persists full call context
Angry / Confused Patients	6/10	Layer 2 escalation + graceful handoff to human receptionist
WhatsApp API Approval	5/10	Apply for Meta Business API early; use Twilio SMS as fallback

8 Suggested Extra Features

These additions elevate the project from a voice booking tool to a full clinic management platform:

High Impact

Re-engagement WhatsApp for Inactive Patients
Analytics agent flags patients inactive for 6+ months
Sends automated WhatsApp: "██████████ ██████████ ██████████ X ██████████ ██████████ — ██████████ ██████████"
Value: Directly generates revenue from the analytics pipeline
No-Show Follow-Up Flow
When secretary marks patient as no_show → auto WhatsApp triggers
"██████████ ██████████ ██████████ ██████████ ██████████ ██████████"
Value: Closes the patient lifecycle loop — turns no-shows into future revenue
Call Recording + AI Summary
Save Twilio recording → Whisper transcription → GPT-4o summarization
Auto-appended to patient record as a note
Example: "Patient called about knee pain — booked Thursday 3PM Dr. Ahmed"
Value: Automatic medical documentation — massive real-world value
HuggingFace Publication (Research Contribution)
Publish Egyptian Arabic clinic TTS dataset on HuggingFace Hub
Publish fine-tuned SparkTTS model weights
Write benchmark comparison: your model vs Azure ar-EG (MOS scores)
Value: First-of-its-kind Egyptian Arabic dental TTS dataset — gets cited

Medium Impact

Smart Slot Suggestion
If requested doctor is fully booked → agent automatically offers

nearest available slot from same specialty doctor
Value: Shows proactive AI behavior, not just reactive

Multi-Language Toggle
Detect patient language from first utterance (Arabic vs English)
Switch agent persona and TTS voice accordingly
Value: Covers expat patients in Cairo clinics

Patient Authentication
Ask patient to confirm name or date of birth before accessing their data
Prevents wrong-person appointment changes
Value: Shows security thinking in AI system design

Installment Payment Tracking
Patients paying in installments tracked in payments table
Analytics agent flags overdue installments monthly
Agent can send payment reminder WhatsApp automatically
Value: Dentist clinics commonly use installment plans in Egypt

9 Implementation Roadmap

Phase	Week	Goal	Deliverable
Phase 1	Week 1-2	Dataset Creation + SparkTTS Fine-tuning	Egyptian Arabic TTS model on HuggingFace
Phase 2	Week 3	LangGraph Agent 1 + Tools + DB Schema	Working voice receptionist agent
Phase 3	Week 4	Whisper STT + Twilio Integration	End-to-end working phone call
Phase 4	Week 5	Agent 2 — Doctor Recording Pipeline	Voice procedure extraction + pricing
Phase 5	Week 6	Agent 4 — WhatsApp Booking Channel	Text-based booking via WhatsApp
Phase 6	Week 7	Reminders + WhatsApp Business API	Automated day-before reminder system
Phase 7	Week 8	Agent 3 — Analytics + React Dashboard	Monthly reports + live charts
Phase 8	Week 9	Polish + Demo Video + Publish	Portfolio-ready demo + GitHub

CV Impact

Dimension	Before	After
Agentic AI	None	Four-agent system with clear separation of concerns
Voice AI	Whisper in RAG only	Full production voice pipeline (STT + TTS)
Custom ML Model	None	Fine-tuned SparkTTS on Egyptian Arabic
Multi-channel AI	None	Voice + WhatsApp booking — same brain, two channels
Real-time Systems	None	Scheduling, reminders, live dashboard
Open Source	None	HuggingFace dataset + model publication
End-to-end Product	APIs only	Full platform: voice + WhatsApp + analytics + UI
CV Rating	6.8 / 10	9.4 / 10